

Documento editable - Método de la Falsa Posición

Estudiante: Andrey Felipe Pinto Uribe

Código: 0192177

Archivo fuente: Metodo_Falsa_Posicion_0192177_Andrey_Felipe_Pinto_Uribe.py

Link en github: https://github.com/pintodeveloper/numerical-analysis/tree/main/entregas_moodle/falsa_posicion

1. Objetivo

El programa implementa el método de la falsa posición para aproximar una raíz de una función continua usando un intervalo inicial con cambio de signo.

2. Funcionamiento general del programa

- La tabla muestra el intervalo actual, la aproximación c calculada por falsa posición, $f(c)$ y el error.
- El método actualiza el extremo correspondiente según el signo de $f(c)$.
- El error se calcula como la diferencia entre aproximaciones consecutivas.
- La verificación $f(\text{raíz})$ permite comprobar que el resultado está cerca de una raíz real.

3. Pruebas o validaciones realizadas

- Se ejecuta con $f(x) = x^3 - x - 2$ en el intervalo $[1, 2]$.
- Se comprueba que exista cambio de signo antes de iniciar.
- Se revisan las iteraciones, la raíz aproximada y el valor de la función en la raíz.

4. Evidencia del código fuente

A continuación se incluye el código fuente del archivo entregado. Este contenido es editable en Word.

```
1  """
2  Método de la Falsa Posición
3  Estudiante: Andrey Felipe Pinto Uribe
4  Código: 0192177
5
6  Archivo preparado para entrega en Moodle. Contiene la implementación,
7  un ejemplo de ejecución y validaciones básicas de entrada.
8  """
9
10 from dataclasses import dataclass
11 from typing import Callable, List
12
13
```

```

14 @dataclass
15 class IteracionFalsaPosicion:
16     numero: int
17     a: float
18     b: float
19     c: float
20     fc: float
21     error: float
22
23
24 @dataclass
25 class ResultadoFalsaPosicion:
26     raiz: float
27     iteraciones: int
28     convergio: bool
29     historial: List[IteracionFalsaPosicion]
30
31
32 def falsa_posicion(
33     funcion: Callable[[float], float],
34     a: float,
35     b: float,
36     tolerancia: float = 1e-6,
37     max_iter: int = 100,
38 ) -> ResultadoFalsaPosicion:
39     if a >= b:
40         raise ValueError("El extremo izquierdo debe ser menor que el derecho.")
41     if tolerancia <= 0:
42         raise ValueError("La tolerancia debe ser positiva.")
43     if max_iter <= 0:
44         raise ValueError("El número máximo de iteraciones debe ser positivo.")
45
46     fa = funcion(a)
47     fb = funcion(b)
48
49     if fa == 0:

```

```

50         return ResultadoFalsaPosicion(raiz=a, iteraciones=0, convergio=True, historial=[])
51     if fb == 0:
52         return ResultadoFalsaPosicion(raiz=b, iteraciones=0, convergio=True, historial=[])
53     if fa * fb > 0:
54         raise ValueError("La función debe cambiar de signo en el intervalo [a, b].")
55
56     historial: List[IteracionFalsaPosicion] = []
57     c_anterior = None
58
59     for numero in range(1, max_iter + 1):
60         c = b - fb * (b - a) / (fb - fa)
61         fc = funcion(c)
62         error = abs(c - c_anterior) if c_anterior is not None else abs(b - a)
63
64         historial.append(
65             IteracionFalsaPosicion(
66                 numero=numero,
67                 a=a,
68                 b=b,
69                 c=c,
70                 fc=fc,
71                 error=error,
72             )
73         )
74
75         if fc == 0 or error <= tolerancia:
76             return ResultadoFalsaPosicion(
77                 raiz=c,
78                 iteraciones=numero,
79                 convergio=True,
80                 historial=historial,
81             )
82
83         if fa * fc < 0:
84             b = c
85             fb = fc

```

```

86         else:
87             a = c
88             fa = fc
89
90             c_anterior = c
91
92         return ResultadoFalsaPosicion(
93             raiz=historial[-1].c,
94             iteraciones=max_iter,
95             convergio=False,
96             historial=historial,
97         )
98
99
100 def imprimir_tabla(historial: List[IteracionFalsaPosicion]) -> None:
101     encabezado = f"{'i':>3} {'a':>12} {'b':>12} {'c':>12} {'f(c)':>14} {'error':>12}"
102     print(encabezado)
103     print("-" * len(encabezado))
104     for paso in historial:
105         print(
106             f"{paso.numero:>3} "
107             f"{paso.a:>12.6f} "
108             f"{paso.b:>12.6f} "
109             f"{paso.c:>12.6f} "
110             f"{paso.fc:>14.6e} "
111             f"{paso.error:>12.6e}"
112         )
113
114
115 def funcion_ejemplo(x: float) -> float:
116     return x**3 - x - 2
117
118
119 def main() -> None:
120     resultado = falsa_posicion(funcion_ejemplo, 1.0, 2.0)
121     print("Método de la Falsa Posición")

```

```

122     imprimir_tabla(resultado.historial)
123     print(f"\nRaiz aproximada: {resultado.raiz:.10f}")
124     print(f"Iteraciones: {resultado.iteraciones}")
125     print(f"Verificacion f(raiz): {funcion_ejemplo(resultado.raiz):.10e}")
126
127
128 if __name__ == "__main__":
129     main()

```

5. Evidencia de ejecución

La siguiente salida fue obtenida ejecutando el archivo Python de esta carpeta.

python Metodo_Falsa_Posicion_0192177_Andrey_Felipe_Pinto_Uribe.py

Método de la Falsa Posición

i	a	b	c	f(c)	error

1	1.000000	2.000000	1.333333	-9.629630e-01	1.000000e+00
2	1.333333	2.000000	1.462687	-3.333389e-01	1.293532e-01
3	1.462687	2.000000	1.504019	-1.018180e-01	4.133244e-02
4	1.504019	2.000000	1.516331	-2.989480e-02	1.231156e-02
5	1.516331	2.000000	1.519919	-8.675066e-03	3.587985e-03
6	1.519919	2.000000	1.520957	-2.508805e-03	1.038931e-03
7	1.520957	2.000000	1.521258	-7.248228e-04	3.002678e-04
8	1.521258	2.000000	1.521344	-2.093499e-04	8.673511e-05
9	1.521344	2.000000	1.521370	-6.046133e-05	2.505031e-05
10	1.521370	2.000000	1.521377	-1.746113e-05	7.234550e-06
11	1.521377	2.000000	1.521379	-5.042711e-06	2.089317e-06
12	1.521379	2.000000	1.521379	-1.456314e-06	6.033863e-07

Raiz aproximada: 1.5213794618

Iteraciones: 12

Verificacion f(raiz): -1.4563135953e-06

```
from dataclasses import dataclass
from typing import Callable, List

@dataclass
class IteracionFalsaPosicion:
```

i	a	b	c	f(c)	error
1	1.000000	2.000000	1.333333	-9.629630e-01	1.000000e+00
2	1.333333	2.000000	1.462687	-3.333389e-01	1.293532e-01
3	1.462687	2.000000	1.504019	-1.018180e-01	4.133244e-02
4	1.504019	2.000000	1.516331	-2.989480e-02	1.231156e-02
5	1.516331	2.000000	1.519919	-8.675066e-03	3.587985e-03
6	1.519919	2.000000	1.520957	-2.508805e-03	1.038931e-03
7	1.520957	2.000000	1.521258	-7.248228e-04	3.002678e-04
8	1.521258	2.000000	1.521344	-2.093499e-04	8.473511e-05
9	1.521344	2.000000	1.521370	-6.046133e-05	2.505031e-05
10	1.521370	2.000000	1.521377	-1.746113e-05	7.234500e-06
11	1.521377	2.000000	1.521379	-5.042711e-06	2.089317e-06
12	1.521379	2.000000	1.521379	-1.456314e-06	6.033863e-07

Raíz aproximada: 1.5213794618
Iteraciones: 12
Verificación f(raíz): -1.4563135953e-06

Process finished with exit code 0

6. Explicación de la salida

La salida confirma que el programa se ejecuta sin errores. Los valores impresos corresponden al ejemplo de prueba definido en el método ``main``.

- La tabla muestra el intervalo actual, la aproximación c calculada por falsa posición, $f(c)$ y el error.
- El método actualiza el extremo correspondiente según el signo de $f(c)$.
- El error se calcula como la diferencia entre aproximaciones consecutivas.
- La verificación $f(\text{raíz})$ permite comprobar que el resultado está cerca de una raíz real.

7. Conclusión

El programa cumple el objetivo del método numérico, presenta una estructura clara, valida condiciones importantes de entrada y muestra resultados coherentes para el caso de prueba utilizado.